L'ÉDITION MAGAZINE PLATFORM • API INTEGRATION SPECIFICATION

Frontend Developer Guide: Article Photo Flow

Cover Photo & Multi-Photo Gallery Upload, Update, Delete, and Display Specification (Admin, Author & Public APIs)

1. Architecture Overview

Each Article in the platform supports **two distinct photo structures**:

Photo Type	Database Storage	Returned In	Frontend Usage
1. Cover Photo (Hero)	`Tbl_Article.PhotoUrl` `PhotoCaption`, `PhotoCredit`	**Article List** & **Article Detail**	Fast card feed rendering on Home/Category pages + main Hero Banner on Article Detail page.
2. Gallery Photos (Multi-Photo)	`Tbl_ArticlePhoto` (`ArticlePhotoId`, `PhotoUrl`, `Caption`, `Credit`, `SortOrder`)	**Article Detail** (`data.photos[]`)	Ordered photo gallery / carousel inside the article reading view.

> **Why `multipart/form-data` + Relative URLs (No Base64)?**
>
> - **Uploads (Create / Update)** use standard `FormData` (`multipart/form-data`). Do **not** convert images to Base64 strings.
> - **Responses (List / Detail)** return lightweight relative URLs (e.g. `/uploads/articles/2026/10/01JAPHT0000000000000000001.webp`) or external URLs (`https://...`).
> - **Browser Caching**: Uploaded files are named with immutable 26-char ULIDs and served with `Cache-Control: public, max-age=31536000, immutable` .

2. Client-Side Validation Rules & File Limits

Validation Rule	Allowed Values / Limit	Backend Error Response
Allowed Extensions & MIME	`.jpg`, `.jpeg`, `.png`, `.webp` (`image/jpeg`, `image/png`, `image/webp`)	`400 Bad Request`
Binary Magic Bytes	Verified server-side against JPEG, PNG, and RIFF/WEBP headers	`400 Bad Request` (Blocks renamed files)
Max Single Image Size	**5 MB** (`5,242,880` bytes) per image	`400 Bad Request`
Max Gallery Photos / Call	**10 gallery photos** per Create or Update request (**64 MB** total body cap)	`400 Bad Request` / `413`

3. API Endpoints Summary

API Portal	Action	HTTP Method & Route	Payload Format
Admin API	Create / Update Article + Photos	`POST /api/admin/article/Create` & `.../Update`	`multipart/form-data`
Admin API	Get Detail (with Gallery) / List	`POST /api/admin/article/Detail` & `.../List`	`application/json`
Author API	Create / Update Draft + Photos	`POST /api/author/article/Create` & `.../Update`	`multipart/form-data`
Author API	Get My Article Detail / List	`POST /api/author/article/Detail` & `.../List`	`application/json`
Public API	Get Published List / Detail by Slug	`GET /api/public/article/list` & `GET .../article/{slug}`	Query / Route Param

4. Flow 1: Creating an Article with Photos (`POST .../Create`)

FormData Key	Type	Required	Description
`Title` / `SubTitle` / `Body`	`string`	**Title Req**	Headline (max 200), Subtitle (max 400), and HTML Body
`ArticleCategoryId` / `AuthorId`	`string`	**Required**	26-character ULIDs for Category and Author
`Status` / `IsFeatured` / `IsSpotlight`	`string/bool`	**Optional**	`"Draft"` or `"Published"` ; booleans as `"true"` / `"false"`
`CoverPhoto`	`File`	**Optional**	Binary cover image file (`.jpg` , `.png` , `.webp` , max 5 MB)
`PhotoUrl` / `PhotoCaption` / `PhotoCredit`	`string`	**Optional**	Fallback URL (if `CoverPhoto` omitted), Caption (200), Credit (100)
`GalleryItems[i].File`	`File`	**Optional**	Gallery image file at index `i` (`0, 1, 2...`)
`GalleryItems[i].Caption` / `.Credit` / `.SortOrder`	`string/int`	**Optional**	Caption, Credit, and SortOrder (`1, 2, 3...`) paired with `GalleryItems[i].File`

TypeScript / React Implementation (Create Article)

```
export interface GalleryUploadItem {
  file: File;
  caption?: string;
  credit?: string;
  sortOrder: number;
}

export interface CreateArticleFormInput {
  title: string;
  subTitle?: string;
  body?: string;
  articleCategoryId: string;
  authorId: string;
  status?: "Draft" | "Published";
  isFeatured?: boolean;
  isSpotlight?: boolean;
  coverPhoto?: File | null;
  photoUrl?: string;
  photoCaption?: string;
  photoCredit?: string;
  galleryItems?: GalleryUploadItem[];
}

export async function createArticle(apiBaseUrl: string, input: CreateArticleFormInput, jwtToken?: string) {
  const formData = new FormData();

  formData.append("Title", input.title);
  if (input.subTitle) formData.append("SubTitle", input.subTitle);
  if (input.body) formData.append("Body", input.body);
  formData.append("ArticleCategoryId", input.articleCategoryId);
  formData.append("AuthorId", input.authorId);
  formData.append("Status", input.status ?? "Draft");
  formData.append("IsFeatured", String(Boolean(input.isFeatured)));
  formData.append("IsSpotlight", String(Boolean(input.isSpotlight)));

  if (input.coverPhoto) {
    formData.append("CoverPhoto", input.coverPhoto);
  } else if (input.photoUrl) {
    formData.append("PhotoUrl", input.photoUrl);
  }
  if (input.photoCaption) formData.append("PhotoCaption", input.photoCaption);
  if (input.photoCredit) formData.append("PhotoCredit", input.photoCredit);

  // Structured Gallery Items (Indexed 0..N to prevent caption misalignment)
  input.galleryItems?.forEach((item, index) => {
    formData.append(`GalleryItems[${index}].File`, item.file);
    formData.append(`GalleryItems[${index}].Caption`, item.caption ?? "");
    formData.append(`GalleryItems[${index}].Credit`, item.credit ?? "");
    formData.append(`GalleryItems[${index}].SortOrder`, String(item.sortOrder));
  });

  // DO NOT manually set 'Content-Type': 'multipart/form-data' header!
  const response = await fetch(`${apiBaseUrl}/api/admin/article/Create`, {
    method: "POST",
    headers: jwtToken ? { Authorization: `Bearer ${jwtToken}` } : undefined,
    body: formData,
  });

  return await response.json();
}
```

5. Flow 2: Updating an Article & Gallery Photos (`POST .../Update`)

FormData Key	Type	Required	Description
`ArticleId`	`string`	**Required**	26-character ULID of the article being updated
`Title` , `SubTitle` , `Body` , `ArticleCategoryId` , `AuthorId`	`string`	**Required**	Core article text & 26-char ULID foreign keys
`CoverPhoto` vs. `PhotoUrl`	`File/str`	**Optional**	Send `CoverPhoto` file to replace cover; or send existing `PhotoUrl` string to preserve current cover
`NewGalleryItems[i].File` / `.Caption` / `.Credit` / `.SortOrder`	`File/str`	**Optional**	New gallery photos to append to the article (indexed `0, 1, 2...`)
`RemoveArticlePhotoIds[i]`	`string`	**Optional**	26-char `ArticlePhotoId` ULIDs of existing gallery photos to soft-delete

TypeScript / React Implementation (Update Article)

```
export async function updateArticle(
  apiBaseUrl: string,
  input: {
    articleId: string;
    title: string;
    subTitle?: string;
    body?: string;
    articleCategoryId: string;
    authorId: string;
    newCoverPhoto?: File | null;
    existingPhotoUrl?: string;
    photoCaption?: string;
    photoCredit?: string;
    newGalleryItems?: GalleryUploadItem[];
    removeArticlePhotoIds?: string[];
  },
  jwtToken?: string
) {
  const formData = new FormData();

  formData.append("ArticleId", input.articleId);
  formData.append("Title", input.title);
  if (input.subTitle) formData.append("SubTitle", input.subTitle);
  if (input.body) formData.append("Body", input.body);
  formData.append("ArticleCategoryId", input.articleCategoryId);
  formData.append("AuthorId", input.authorId);

  if (input.newCoverPhoto) {
    formData.append("CoverPhoto", input.newCoverPhoto);
  } else if (input.existingPhotoUrl) {
    formData.append("PhotoUrl", input.existingPhotoUrl);
  }
  if (input.photoCaption) formData.append("PhotoCaption", input.photoCaption);
  if (input.photoCredit) formData.append("PhotoCredit", input.photoCredit);

  input.newGalleryItems?.forEach((item, index) => {
    formData.append(`NewGalleryItems[${index}].File`, item.file);
    formData.append(`NewGalleryItems[${index}].Caption`, item.caption ?? "");
    formData.append(`NewGalleryItems[${index}].Credit`, item.credit ?? "");
    formData.append(`NewGalleryItems[${index}].SortOrder`, String(item.sortOrder));
  });

  input.removeArticlePhotoIds?.forEach((photoId, index) => {
    formData.append(`RemoveArticlePhotoIds[${index}]`, photoId);
  });

  const response = await fetch(`${apiBaseUrl}/api/admin/article/Update`, {
    method: "POST",
    headers: jwtToken ? { Authorization: `Bearer ${jwtToken}` } : undefined,
    body: formData,
  });

  return await response.json();
}
```

6. Flow 3: Displaying Photos on the Frontend (`Show` Flow)

JSON Response Structure (`ArticleResModel`)

```
{
  "isSuccess": true,
  "isError": false,
  "responseType": 0,
  "message": "Article retrieved successfully.",
  "data": {
    "articleId": "01JART0000000000000000001",
    "title": "The Art of Quiet Luxury",
    "slug": "the-art-of-quiet-luxury",
    "photoUrl": "/uploads/articles/2026/10/01JCOVER00000000000000001.webp",
    "photoCaption": "Morning light across raw linen and travertine.",
    "photoCredit": "Photo by L'ÉDITION Studio",
    "photos": [
      {
        "articlePhotoId": "01JAPHT0000000000000000001",
        "articleId": "01JART0000000000000000001",
        "photoUrl": "/uploads/articles/2026/10/01JAPHT0000000000000000001.webp",
        "caption": "Handcrafted stoneware vessels in afternoon shadow.",
        "credit": "Photo by L'ÉDITION Studio",
        "sortOrder": 1,
        "createdAt": "2026-10-10T13:30:00+00:00"
      }
    ]
  }
}
```

Frontend URL Helper (`resolveImageUrl`)

```
const API_BASE_URL = process.env.NEXT_PUBLIC_API_BASE_URL ?? "http://localhost:5222";

export function resolveImageUrl(
  photoUrl: string | null | undefined,
  fallbackUrl = "/images/placeholder-editorial.jpg"
): string {
  if (!photoUrl || photoUrl.trim() === "") return fallbackUrl;

  if (
    photoUrl.startsWith("http://") ||
    photoUrl.startsWith("https://") ||
    photoUrl.startsWith("blob:") ||
    photoUrl.startsWith("data:")
  ) {
    return photoUrl;
  }

  const normalizedBase = API_BASE_URL.replace(/\/+$/, "");
  const normalizedPath = photoUrl.startsWith("/") ? photoUrl : `/${photoUrl}`;
  return `${normalizedBase}${normalizedPath}`;
}
```

> **3 Golden Rules for Frontend Developers**
>
> 1. **Never manually set `Content-Type: multipart/form-data` in `fetch` headers** — let the browser set it automatically so the `boundary=----...` token is included.
> 2. **Always use 0-based sequential indices (`0, 1, 2...`)** for `GalleryItems[i]` , `NewGalleryItems[i]` , and `RemoveArticlePhotoIds[i]` .
> 3. **When editing an article without uploading a new Cover Photo**, pass the existing `article.photoUrl` in `formData.append("PhotoUrl", article.photoUrl)` so the current cover photo is preserved.